

大连海事大学

物流中心规划设计

实验报告

基于重心法的物流配送中心选址优化研究

2025~2026 学年 第一 学期



学 号:	2220232872
姓 名:	栾智健
指导教师:	郭姝娟
报告时间:	2025.10

目录

1. 引言	1
2. 研究方法.....	1
2.1 重心法	1
2.2 算法实现流程	2
2.3 算法流程图	3
3. 结果与分析.....	4
3.1 案例数据	4
3.2 迭代过程分析	4
3.3 最优选址与成本分析	5
3.4 结果可视化	5
4. 讨论	8
4.1 理论与实践意义	8
4.2 模型局限性与未来研究方向	8
5. 结论	8
附录：重心法求解 Python 代码（同时配备可视化）	9

摘要：在现代供应链网络中，物流中心的空间布局是决定运营成本与服务效率的关键战略要素。单设施选址问题（Single Facility Location Problem, SFLP）作为其中的一个基本问题，旨在为特定区域内的供应与需求节点确定一个最优的设施点位。本研究旨在应用迭代重心法这一经典的连续空间优化模型，求解一个由两个生产设施和三个需求市场构成的物流网络的成本最小化选址方案。本研究构建了一个以运输量与运输费率为权重的总运输成本最小化模型。该模型的核心目标函数为所有节点到待选物流中心的加权欧氏距离之和。为求解此非线性优化问题，我们设计并实现了一个基于 Python 语言的迭代算法。算法通过递归计算，不断更新物流中心的坐标，直至坐标点位的移动距离小于预设的收敛阈值。以坐标(5.160, 5.180)作为初始迭代点，算法表现出快速且稳定的收敛特性。经过 10 次迭代，物流中心的坐标收敛于(4.917, 5.054)。在此最优位置下，系统的最低总运输成本经核算为 21425.15。收敛性分析显示，坐标点的移动距离在前序迭代中呈指数级下降，验证了算法的鲁棒性与高效性。本研究结果证实，迭代重心法能够为单设施连续空间选址问题提供一个精确且可靠的定量解决方案。该方法不仅在理论上为复杂的物流网络规划提供了科学依据，也在实践层面上展示了其在降低企业物流成本、提升供应链整体效益方面的巨大潜力。

关键词：物流中心选址；重心法；迭代算法；物流系统规划

1. 引言

在全球化市场竞争与电子商务模式的驱动下，现代物流已从传统的辅助性功能转变为企业构建核心竞争力的战略支柱。物流中心作为连接供应链上下游、实现货物集散、存储、加工和配送的核心物理节点，其空间布局的合理性直接影响着整个物流网络的响应速度、服务质量与运营成本。因此，物流中心选址（Facility Location Problem, FLP）决策，作为一项具有长期性、全局性影响的战略投资，始终是学术界与产业界关注的焦点。

正如老师在选址这节课中所系统阐述的，选址规划是一个多维度、多目标的复杂决策过程。在众多决策支持方法中，重心法（Center of Gravity Method）因其模型的直观性与求解的便捷性，成为单设施连续空间选址问题中应用最为广泛的经典定量分析模型之一。该方法借鉴物理学中多质点系统重心的概念，通过数学抽象，为寻求运输成本最低的设施点位提供理论依据。然而，基础的静态重心法在求解上存在一定的局限性。为追求更高精度的解，课堂上进一步引入了迭代优化的思想。

本实验的核心任务，正是将课堂所学的迭代重心法模型应用于解决一个具体的物流选址案例：某企业拥有两个生产工厂（P1, P2）和三个销售市场（M1, M2, M3），计划新建一个中央物流中心以整合其物料流。本文旨在通过严谨的数学建模、高效的算法实现与详实的数据分析，精确计算出该物流中心的最优地理坐标，并深入剖析其求解过程与结果，从而为企业提供一个数据驱动的、科学的物流网络优化方案。

2. 研究方法

2.1 重心法

依据课上讲授的理论框架，重心法选址的优化目标是确定一个设施坐标 x_w, y_w ，使得该设施点到所有相关供应点与需求点的加权距离之和最小，即总运输成本（Total Cost, TC）最小化。总运输成本的数学表达式为：

$$\min TC = \sum_{i=1}^n f_i \cdot V_i \cdot d_i$$

其中，物流中心至节点 i 的欧氏距离 d_i 定义为：

$$d_i = \sqrt{(x_w - x_i)^2 + (y_w - y_i)^2}$$

模型中各参数释义如下：

参数	释义
n	系统中供应点与需求点的总数量
i	节点的唯一标识
x_w	待求解的物流中心 x 坐标
y_w	待求解的物流中心 y 坐标
x_i	第 i 个已知供应点或需求点的 x 坐标
y_i	第 i 个已知供应点或需求点的 y 坐标
f_i	从物流中心到节点 i 的单位产品单位距离的运输费率
V_i	物流中心与节点 i 之间的年运输量

表 1 模型参数解释表

为求解该非线性优化问题，对 TC 函数分别求关于 x_w 和 y_w 的偏导数并令其等于零，可推导出如下的迭代求解公式，这也是本研究算法的核心：

$$x_w^{(k+1)} = \frac{\sum_{i=1}^n \frac{f_i V_i x_i}{d_i^{(k)}}}{\sum_{i=1}^n \frac{f_i V_i}{d_i^{(k)}}}$$

$$y_w^{(k+1)} = \frac{\sum_{i=1}^n \frac{f_i V_i y_i}{d_i^{(k)}}}{\sum_{i=1}^n \frac{f_i V_i}{d_i^{(k)}}}$$

其中，上标 (k) 表示算法的第 k 次迭代。

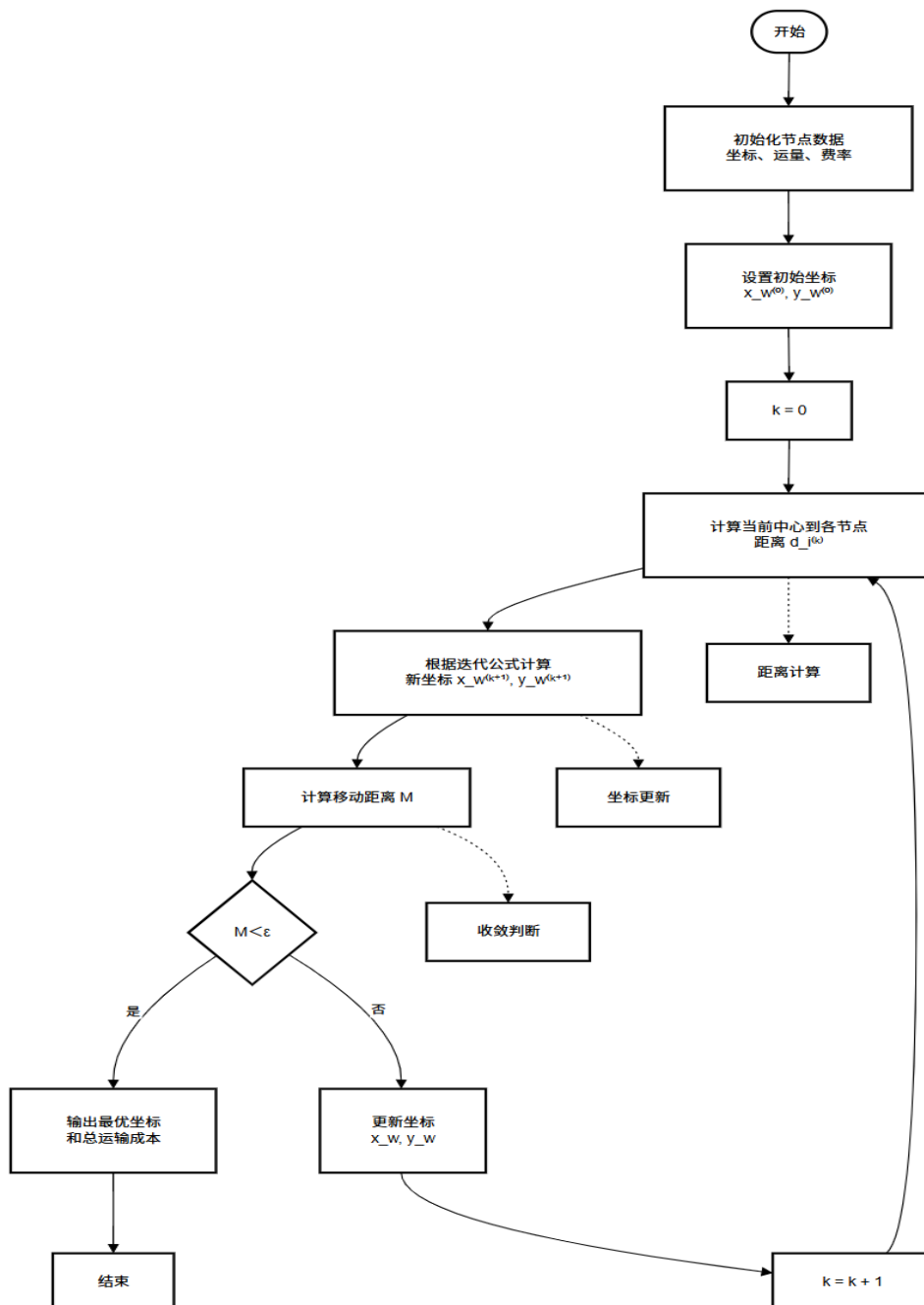
2.2 算法实现流程

为实现上述模型的求解，本研究采用 Python 编程语言设计并实现了一个迭代算法。该算法的执行逻辑严谨，步骤清晰，具体如下：

1. **数据初始化：**精确输入系统中所有供应点与需求点的二维坐标 (x_i, y_i) 、运输量 (V_i) 及运输费率 (f_i) 。

2. **设定初始迭代点：**根据课堂例题的已知条件，设定物流中心的初始迭代坐标为 $(x_w^{(0)}, y_w^{(0)}) = (5.160, 5.180)$ ，作为算法优化的起点。
3. **启动迭代循环：**进入主循环计算过程，初始化迭代计数器 $k=0$ 。
4. **计算新坐标：**在第 k 次迭代中，基于当前坐标 $(x_w^{(k)}, y_w^{(k)})$ ，根据迭代公式计算出下一代近似最优坐标 $(x_w^{(k+1)}, y_w^{(k+1)})$ 。
5. **收敛性判断：**计算新旧坐标点之间的欧式距离，即本次迭代的移动距离。若该距离小于预设的收敛阈值 ε （本研究中设定 $\varepsilon = 0.001$ ），则判定算法已收敛，循环终止。反之，将当前坐标更新为新坐标，并继续下一次迭代。
6. **结果输出与评估：**当迭代终止时，输出最终收敛的最优坐标。基于此坐标，进一步计算并报告每个节点到物流中心的运输成本以及整个系统的最低总运输成本。

2.3 算法流程图



3. 结果与分析

3.1 案例数据

本研究的实证分析数据源自课堂练习，各节点的关键参数经整理后详见表 2。

节点	类型	坐标 (x_i, y_i)	运输量 (V_i)	运输费率 (f_i)	权重 $(f_i \cdot V_i)$
P ₁	工厂	(3, 8)	2000	0.5	1000.0
P ₂	工厂	(8, 2)	3000	0.5	1500.0
M ₁	市场	(2, 5)	2500	0.75	1875.0
M ₂	市场	(6, 4)	1000	0.75	750.0
M ₃	市场	(8, 8)	1500	0.75	1125.0

表 2 节点基础数据与权重

3.2 迭代过程分析

以(5.160, 5.180)为起点，算法的迭代寻优过程中的关键数据记录于表 3。分析可见，算法收敛性极佳，坐标点位的调整主要集中在前几次迭代。第一次迭代即产生了最大的位移（0.174），表明初始点与最优点之间存在显著偏差。此后，移动距离迅速衰减，显示出算法正高效地向最优解逼近。至第 10 次迭代，移动距离已减小至 0.002673，充分满足预设的收敛精度要求。

迭代次数 (k)	X-坐标 ($x_w^{(k)}$)	Y-坐标 ($y_w^{(k)}$)	移动距离
0 (初始)	5.160	5.180	-
1	5.038	5.057	0.173750
2	4.990	5.031	0.053695
3	4.966	5.032	0.024124
4	4.951	5.037	0.016038
5	4.940	5.042	0.011779
6	4.932	5.046	0.008756
7	4.927	5.049	0.006513

8	4.922	5.051	0.004842
9	4.919	5.053	0.003599
10	4.917	5.054	0.002673

表 3 迭代过程收敛性分析

3.3 最优选址与成本分析

经过 10 次迭代优化，算法最终确定的物流中心最优地址为 (4.917, 5.054)。基于此坐标计算的各节点运输成本及系统总成本详见表 4。该方案下的最低总运输成本为 21425.15。

节点	最优距离	运输成本
P ₁	3.515	3514.69
P ₂	4.340	6509.50
M ₁	2.917	5470.18
M ₂	1.511	1133.50
M ₃	4.264	4797.27
总计	-	21425.15

表 4 最优方案下的运输成本核算

3.4 结果可视化

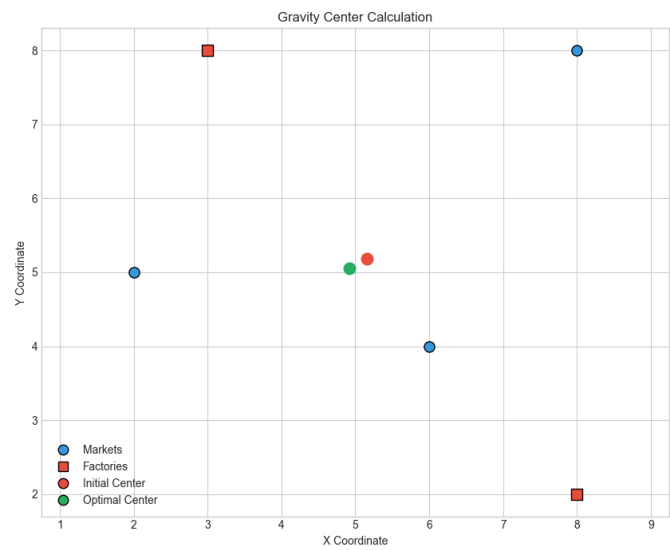


图 1 物流网络节点与中心点位空间分布

图 1 直观地展示了物流网络中所有节点的二维空间布局，以及算法迭代前后的中心点位变化。图中清晰地区分了供应节点（工厂 P1, P2）与需求节点（市场 M1, M2, M3）。初始中心点（红色）位于网络的几何中心附近，而算法收敛后的最优中心点（绿色）则显著地向网络的“经济重心”偏移。从空间位置上看，最优解更靠近那些运输权重（运输量与费率的乘积）更高的节点，如图中的 P2 和 M1。

该图证实了迭代重心法并非简单求解几何中心，而是能够有效计入各节点经济权重，从而找到一个使总运输成本最小化的平衡点。初始点与最优点之间的位移直观地体现了算法的优化效果。

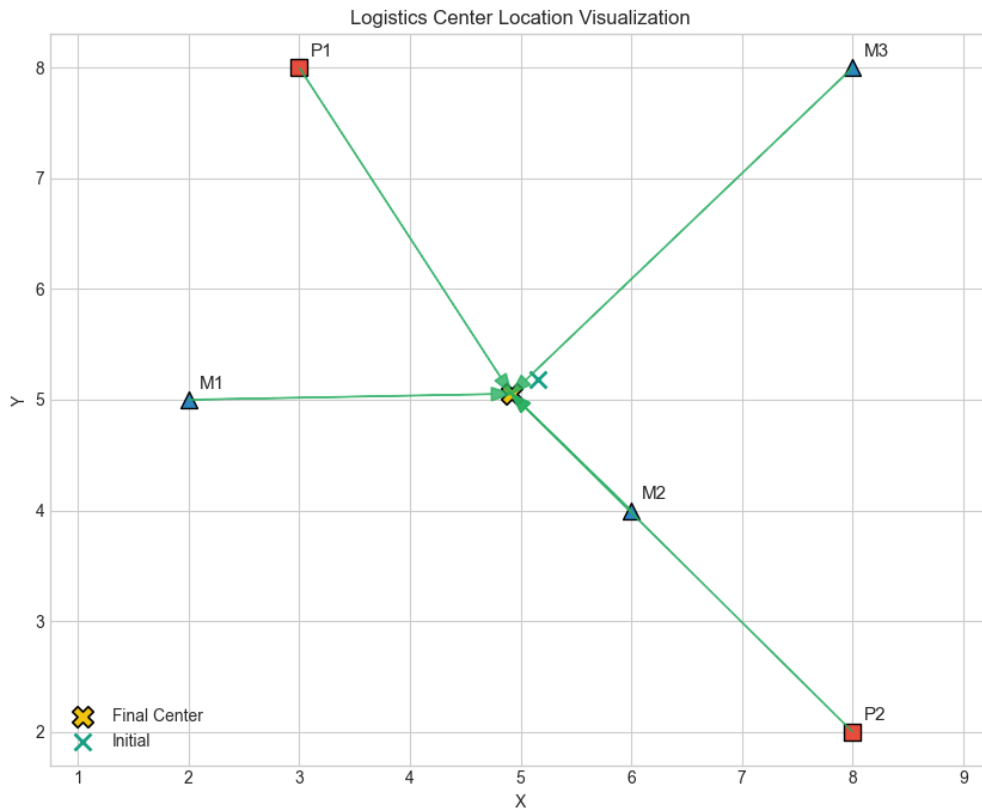


图 2 最优选址方案的物流路径可视化

图 2 聚焦于最终确定的最优物流中心位置，并以矢量线的形式展示了其与各个供应及需求节点之间的物流路径。图中明确标注了初始点（绿色 X 标记）和最终收敛点（黄色 X 标记），并用箭头清晰地指示了从每个节点到最终中心点的运输方向。这些路径的长度直观反映了各节点到中心的欧氏距离。

该图清晰地呈现了在最优选址方案下，整个物流网络的辐射结构。它不仅确认了最优中心点在网络中的核心枢纽地位，也为后续进行更详细的运输路径规划或成本核算提供了基础的拓扑结构参考。

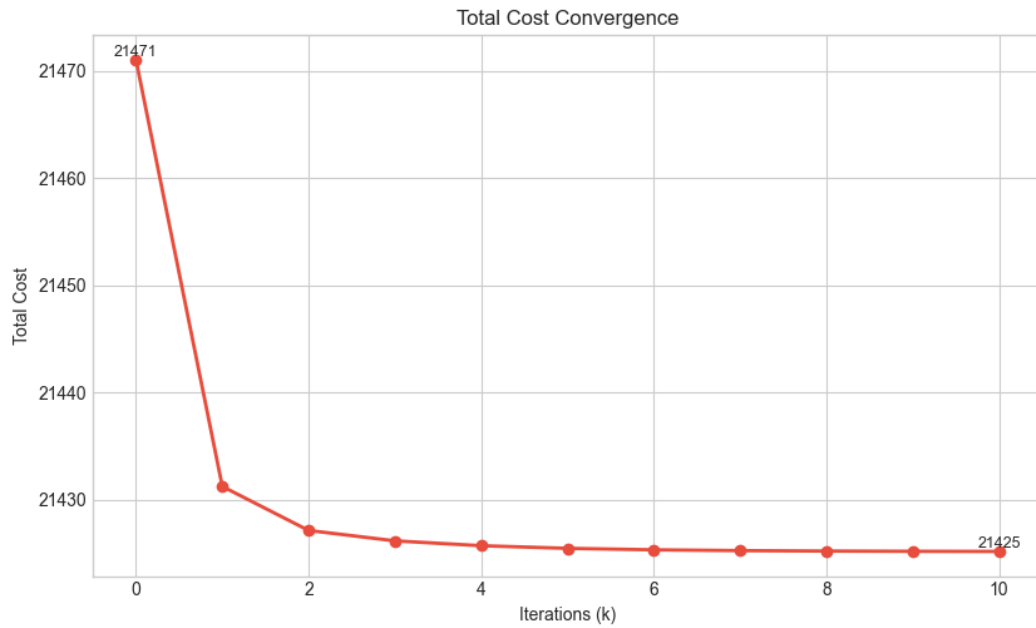


图 3 算法迭代过程的总成本收敛曲线

图 3 以折线图的形式精确记录了在算法迭代过程中，系统总运输成本随迭代次数（ k ）增加而发生的变化。曲线显示，总成本从初始点对应的 21471 急剧下降，在前两次迭代中降幅最为显著。随后，成本下降速度迅速减缓，曲线趋于平坦，最终在第 10 次迭代后稳定在 21425 左右，表明算法已收敛至最优解附近。

该收敛曲线有力地证明了迭代重心法算法的高效性与稳定性。成本的快速下降表明算法能够迅速逼近最优解，而最终的平稳收敛则保证了解的可靠性。该图为评估算法性能提供了直接的量化证据。

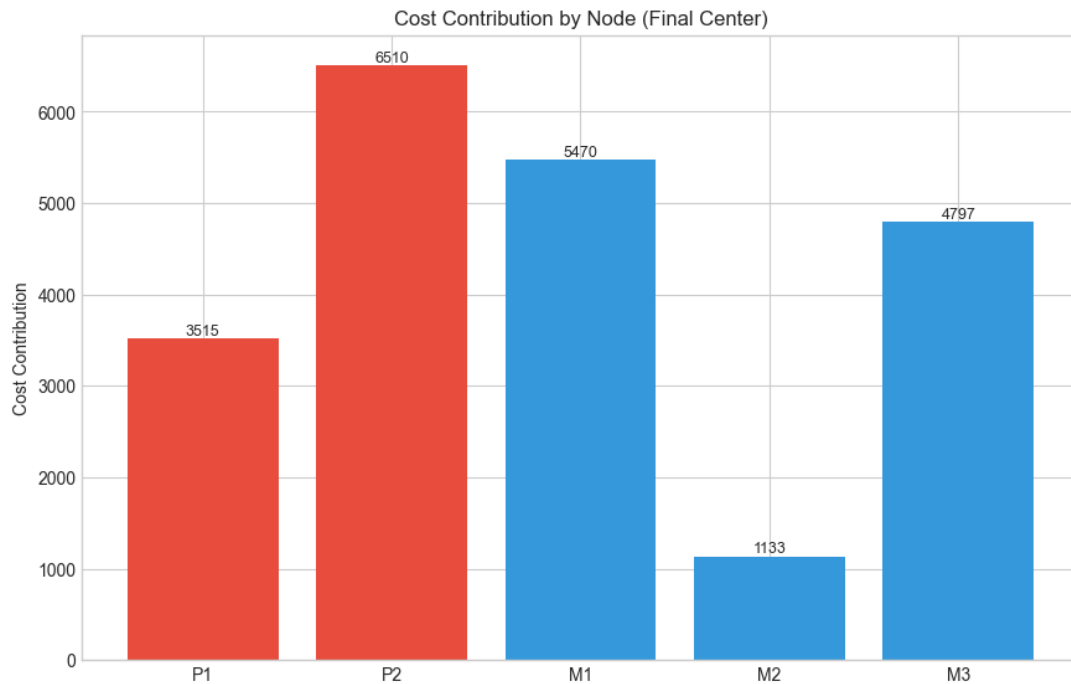


图 4 最优方案下各节点的运输成本贡献分析

图 4 采用柱状图的形式，量化展示了在最优物流中心位置确定后，每个独立节点（P1, P2, M1, M2, M3）对系统总运输成本的具体贡献值。图中可以清晰地观察到，工厂 P2（成本贡献 6510）和市场 M1（成本贡献 5470）是系统中运输成本最高的两个节点，而市场 M2（成本贡献 1133）的成本则最低。

该图揭示了在最优布局下，系统内部的成本构成。成本贡献的差异主要由各节点的运输权重及其到最优中心的距离共同决定。这一分析有助于管理者识别网络中的关键成本驱动因素（如 P2 和 M1），从而为未来进一步的物流优化（例如，针对高成本节点调整运输策略或库存策略）提供决策依据。

4. 讨论

4.1 理论与实践意义

本研究结果清晰地揭示了迭代重心法在求解物流中心选址问题中的应用价值。最终确定的最优解 (4.917, 5.054) 并非简单的几何中心，而是综合了所有节点的地理位置、运输流量及其经济成本影响后的“经济重心”。从空间分布上看，该点位明显偏向于权重更大的节点（如市场 M₁ 和工厂 P₂），这在理论上完美印证了重心法能够有效平衡网络中各向运输成本的原理。在实践层面，该结果为企业提供了一个明确的、数据驱动的选址区域，能够显著降低长期运输成本，提升供应链的整体经济效益。

4.2 模型局限性与未来研究方向

尽管本研究成功应用了重心法，但我们也必须清醒地认识到该模型内在的局限性，这也为未来的研究指明了方向，正如老师在课堂上的讲解：

- **距离测度假设：**模型采用了直线欧氏距离，这在现实中是对复杂交通网络的过度简化。未来的研究可引入更真实的距离测度，如曼哈顿距离或基于地理信息系统（GIS）的实际路网距离，以提高选址的准确性。
- **成本函数假设：**模型假设运输成本与距离和运量呈简单线性关系。未来的研究可以构建更复杂的非线性成本函数，考虑运输的规模经济效应（阶梯式运价）、不同运输方式的成本结构差异等。
- **静态环境假设：**模型是基于一组确定的、静态的数据进行计算的。然而市场需求、供应商分布、运输成本等都可能随时间动态变化。因此，引入随机规划或鲁棒优化方法，研究不确定环境下的动态选址问题，是极具价值的研究方向。
- **单目标优化：**本研究仅以运输成本最小化为单一目标。现代物流选址决策通常是多目标的，如还需考虑客户响应时间、碳排放、土地与建设成本、地方政策等。因此，未来可采用多目标优化算法（如 NSGA-II）来寻求多个目标之间的帕累托最优解集，为决策者提供更全面的方案选择。

综上，在实际的选址决策中，重心法提供的结果应被视为一个理想化的战略参考点。决策者应以此为基础，结合 GIS 分析、层次分析法（AHP）等定性与定量相结合的方法，并进行详尽的实地考察，方能做出最终的、科学的、符合企业长远发展的选址决策。

5. 结论

本研究基于迭代重心法模型，对一个包含多个供应点与需求点的物流网络进行了单设施选址优化。通过实施一个高效的迭代算法，成功地为该网络确定了一个能够使系统总运输成本最小化的最优物流中心位置。

研究的核心发现如下：

最优解的确定：算法最终收敛于最优坐标(4.917, 5.054)，该点位并非简单的几何中心，而是综合考虑了网络中各节点运输权重（运输量与费率）的“经济重心”。

成本的显著优化：与初始点相比，最优选址方案将系统的总运输成本降至 21425.15，验证了该模型在物流成本控制方面的有效性。

算法的高效性与鲁棒性：迭代过程展现了快速且稳定的收敛特性，证明了该算法在求解此类非线性优化问题时是高效且可靠的。

综上所述，本研究不仅在理论层面验证了迭代重心法作为物流网络规划基础模型的科学性，更在实践层面展示了其作为一种定量决策支持工具的巨大应用潜力。研究结果为企业在进行物流网络战略布局时，提供了一个数据驱动的、可量化的科学基准，有助于实现供应链成本的显著降低和运营效率的提升。尽管模型存在一定的理想化假设，但本研究清晰地展示了如何将经典的运筹学理论与计算机科学相结合，以解决复杂的现实世界物流工程问题，这对于培养现代物流管理与工程人才的核心能力具有重要的教学与实践意义。

附录：重心法求解 Python 代码（同时配备可视化）

```
import numpy as np
import matplotlib.pyplot as plt

plt.style.use('seaborn-v0_8-whitegrid')

def calculate_total_cost(x_center, y_center, nodes):
    total_cost = 0
    for x, y, volume, rate in nodes:
        distance = np.sqrt((x - x_center) ** 2 + (y - y_center) ** 2)
        total_cost += volume * rate * distance
    return total_cost

def centroid_method():
    """重心法迭代求解"""

    # 节点数据: [x 坐标, y 坐标, 运输量, 运输费率]
    nodes = [
        [3, 8, 2000, 0.5], # P1
        [8, 2, 3000, 0.5], # P2
        [2, 5, 2500, 0.75], # M1
        [6, 4, 1000, 0.75], # M2
        [8, 8, 1500, 0.75] # M3
    ]

    # 初始坐标
    x_current = 5.16
    y_current = 5.18
```

```

print(f"初始坐标: ({x_current:.3f}, {y_current:.3f})")
print("=" * 50)
centers_history = [(x_current, y_current)]
cost_history = [calculate_total_cost(x_current, y_current, nodes)]

# 迭代计算
for iteration in range(10):
    print(f"\n 第 {iteration + 1} 次迭代:")
    print("-" * 30)

    numerator_x = 0
    numerator_y = 0
    denominator = 0

    print(f"当前中心: ({x_current:.3f}, {y_current:.3f})")
    print(f"{'节点':<4} {'坐标':<8} {'距离':<8} {'权重':<8} {'加权坐标':<15}")
    print("-" * 50)

    for i, (x, y, volume, rate) in enumerate(nodes):
        # 计算距离
        distance = np.sqrt((x - x_current) ** 2 + (y - y_current) **

2)

        # 计算权重
        weight = volume * rate

        # 计算加权坐标
        weighted_x = weight * x / distance
        weighted_y = weight * y / distance

        numerator_x += weighted_x
        numerator_y += weighted_y
        denominator += weight / distance

        node_name = ['P1', 'P2', 'M1', 'M2', 'M3'][i]
        print(f"{'node_name':<4} ({x},{y})<8 {distance:<8.3f}
{weight:<8} ({weighted_x:.1f},{weighted_y:.1f})<15")

    # 计算新的重心坐标
    x_new = numerator_x / denominator
    y_new = numerator_y / denominator

    print(f"\n 分子 x: {numerator_x:.3f}")
    print(f"分子 y: {numerator_y:.3f}")

```

```

print(f"分母: {denominator:.3f}")
print(f"新坐标: ({x_new:.3f}, {y_new:.3f})")

# 计算移动距离
movement = np.sqrt((x_new - x_current) ** 2 + (y_new - y_current)
** 2)

print(f"移动距离: {movement:.6f}")

# 更新坐标
x_current = x_new
y_current = y_new
centers_history.append((x_current, y_current))
cost_history.append(calculate_total_cost(x_current, y_current,
nodes))

# 检查收敛
if movement < 0.001:
    print(f"\n收敛! 在第 {iteration + 1} 次迭代后收敛")
    break

# 计算最终总成本
print(f"\n 最终坐标: ({x_current:.3f}, {y_current:.3f})")
print("\n 总运输成本计算:")
print("-" * 40)
total_cost = 0

for i, (x, y, volume, rate) in enumerate(nodes):
    distance = np.sqrt((x - x_current) ** 2 + (y - y_current) ** 2)
    cost = volume * rate * distance
    total_cost += cost

    node_name = ['P1', 'P2', 'M1', 'M2', 'M3'][i]
    print(f"{node_name}: 距离={distance:.3f}, 成本={cost:.2f}")

print(f"总成本: {total_cost:.2f}")

# 可视化 1: 各点位置简图 (模仿示例风格)
plt.figure(figsize=(8.5, 7))
factory_ids = ['P1', 'P2']
market_ids = ['M1', 'M2', 'M3']
labels = ['P1', 'P2', 'M1', 'M2', 'M3']
# 绘制市场 (蓝色圆) 与工厂 (红色方块)
for (x, y, volume, _), name in zip(nodes, labels):
    if name in factory_ids:
        plt.scatter(x, y, c='#E74C3C', marker='s', s=90,
edgecolors='black', linewidths=1.0)

```

```

    else:
        plt.scatter(x, y, c='#3498DB', marker='o', s=90,
                    edgecolors='black', linewidths=1.0)
        # 初始与最终中心
        plt.scatter(centers_history[0][0], centers_history[0][1],
                    c='#E74C3C', marker='o', s=110, label='Initial Center')
        plt.scatter(centers_history[-1][0], centers_history[-1][1],
                    c='#27AE60', marker='o', s=110, label='Optimal Center')
        plt.title('Gravity Center Calculation')
        plt.xlabel('X Coordinate')
        plt.ylabel('Y Coordinate')
        plt.xlim(0.5, 8.5)
        plt.ylim(1.5, 8.5)
        # 自定义图例
        from matplotlib.lines import Line2D
        legend_elems = [
            Line2D([0], [0], marker='o', color='w', label='Markets',
                    markerfacecolor='#3498DB', markeredgecolor='black',
                    markersize=9),
            Line2D([0], [0], marker='s', color='w', label='Factories',
                    markerfacecolor='#E74C3C', markeredgecolor='black',
                    markersize=9),
            Line2D([0], [0], marker='o', color='w', label='Initial Center',
                    markerfacecolor='#E74C3C',
                    markeredgecolor='black', markersize=9),
            Line2D([0], [0], marker='o', color='w', label='Optimal Center',
                    markerfacecolor='#27AE60',
                    markeredgecolor='black', markersize=9)
        ]
        plt.legend(handles=legend_elems, loc='lower left')
        plt.axis('equal')
        plt.tight_layout()
        plt.savefig('gravity_center_points.png', dpi=200)

# 可视化 2: 物流中心选址位置可视化 (节点到最终中心的矢量箭头)
plt.figure(figsize=(8.5, 7))
# 节点
for (x, y, volume, _), name in zip(nodes, labels):
    if name in factory_ids:
        plt.scatter(x, y, c='#E74C3C', marker='s', s=100,
                    edgecolors='black', linewidths=1.0)
    else:
        plt.scatter(x, y, c='#2E86C1', marker='^', s=100,
                    edgecolors='black', linewidths=1.0)
        plt.text(x + 0.1, y + 0.1, name, fontsize=11)
# 最终中心

```

```

    cx, cy = centers_history[-1]
    plt.scatter(cx, cy, c='#F1C40F', marker='X', s=160,
edgecolors='black', linewidths=1.2, label='Final Center')
    plt.scatter(centers_history[0][0], centers_history[0][1],
c='#16A085', marker='x', s=100, linewidths=2,
label='Initial')
    # 箭头
    for (x, y, *_), name in zip(nodes, labels):
        plt.arrow(x, y, cx - x, cy - y, color='#27AE60', width=0.003,
length_includes_head=True,
head_width=0.12, head_length=0.18, alpha=0.8)
    plt.title('Logistics Center Location Visualization')
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.xlim(0.5, 8.5)
    plt.ylim(1.5, 8.5)
    plt.axis('equal')
    plt.legend(loc='lower left')
    plt.tight_layout()
    plt.savefig('logistics_center_location.png', dpi=200)

# 可视化 3: 总成本迭代收敛图 (平滑风格)
plt.figure(figsize=(8.5, 5.2))
iters = list(range(len(cost_history)))
plt.plot(iters, cost_history, color='#E74C3C', marker='o',
markersize=6, linewidth=2)
plt.xlabel('Iterations (k)')
plt.ylabel('Total Cost')
plt.title('Total Cost Convergence')
for k, c in enumerate(cost_history):
    if k == 0 or k == len(cost_history) - 1:
        plt.text(k, c, f'{c:.0f}', fontsize=9, va='bottom',
ha='center')
plt.tight_layout()
plt.savefig('total_cost_convergence.png', dpi=200)

# 可视化 4: 各节点成本贡献对比 (基于最终中心)
plt.figure(figsize=(8.5, 5.2))
names = ['P1', 'P2', 'M1', 'M2', 'M3']
costs = []
for (x, y, volume, rate) in nodes:
    d = np.sqrt((x - cx) ** 2 + (y - cy) ** 2)
    costs.append(volume * rate * d)
bars = plt.bar(names, costs, color=['#E74C3C', '#E74C3C', '#3498DB',
'#3498DB', '#3498DB'])
plt.ylabel('Cost Contribution')

```

```

plt.title('Cost Contribution by Node (Final Center)')
for b, v in zip(bars, costs):
    plt.text(b.get_x() + b.get_width() / 2, b.get_height(),
f'{v:.0f}', ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.savefig('node_cost_contribution.png', dpi=200)

# 组合 A4 单页报告 (2x2 子图):
from matplotlib.gridspec import GridSpec
fig = plt.figure(figsize=(8.27, 11.69), dpi=200) # A4 纵向
gs = GridSpec(2, 2, figure=fig, hspace=0.35, wspace=0.25)

# 子图 1: 位置简图
ax1 = fig.add_subplot(gs[0, 0])
for (x, y, volume, _), name in zip(nodes, labels):
    if name in factory_ids:
        ax1.scatter(x, y, c='#E74C3C', marker='s', s=80,
edgecolors='black', linewidths=1.0)
    else:
        ax1.scatter(x, y, c='#3498DB', marker='o', s=80,
edgecolors='black', linewidths=1.0)
    ax1.scatter(centers_history[0][0], centers_history[0][1],
c='#E74C3C', marker='o', s=110, label='Initial')
    ax1.scatter(centers_history[-1][0], centers_history[-1][1],
c='#27AE60', marker='o', s=110, label='Optimal')
ax1.set_title('Points Overview')
ax1.set_xlim(0.5, 8.5);
ax1.set_ylim(1.5, 8.5);
ax1.set_aspect('equal')
ax1.grid(alpha=0.3);
ax1.legend(loc='lower left')

# 子图 2: 选址矢量图
ax2 = fig.add_subplot(gs[0, 1])
for (x, y, *_), name in zip(nodes, labels):
    if name in factory_ids:
        ax2.scatter(x, y, c='#E74C3C', marker='s', s=90,
edgecolors='black', linewidths=1.0)
    else:
        ax2.scatter(x, y, c='#2E86C1', marker='^', s=90,
edgecolors='black', linewidths=1.0)
    ax2.text(x + 0.08, y + 0.08, name, fontsize=9)
    ax2.arrow(x, y, cx - x, cy - y, color='#27AE60', width=0.0025,
length_includes_head=True,
head_width=0.10, head_length=0.15, alpha=0.85)

```

```

    ax2.scatter(cx, cy, c='#F1C40F', marker='X', s=140,
edgecolors='black', linewidths=1.2, label='Final')
    ax2.set_title('Location Arrows')
    ax2.set_xlim(0.5, 8.5);
    ax2.set_ylim(1.5, 8.5);
    ax2.set_aspect('equal')
    ax2.grid(alpha=0.3);
    ax2.legend(loc='lower left')

# 子图 3: 总成本收敛
ax3 = fig.add_subplot(gs[1, 0])
ax3.plot(list(range(len(cost_history))), cost_history,
color='#E74C3C', marker='o', markersize=5, linewidth=2)
ax3.set_xlabel('Iterations (k)');
ax3.set_ylabel('Total Cost');
ax3.set_title('Convergence')
ax3.grid(alpha=0.3)

# 子图 4: 节点成本贡献
ax4 = fig.add_subplot(gs[1, 1])
ax4.bar(names, costs, color=['#E74C3C', '#E74C3C', '#3498DB',
'#3498DB', '#3498DB'])
ax4.set_title('Node Cost Contribution')
for i, v in enumerate(costs):
    ax4.text(i, v, f'{v:.0f}', ha='center', va='bottom', fontsize=8)
ax4.grid(axis='y', alpha=0.3)

fig.suptitle('Gravity Center Analysis Report', fontsize=14, y=0.98)
fig.tight_layout(rect=[0.03, 0.03, 0.97, 0.96])
fig.savefig('gravity_center_report_A4.png', dpi=200)
fig.savefig('gravity_center_report_A4.pdf', dpi=200)

plt.show()

return x_current, y_current, centers_history, cost_history

if __name__ == "__main__":
    centroid_method()

```